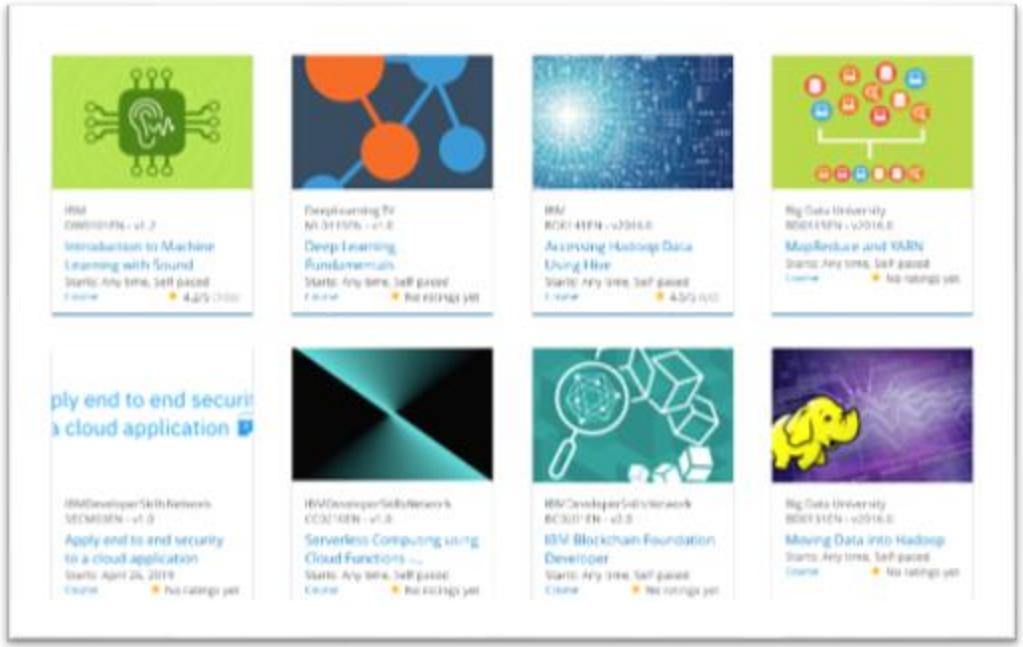
Build a Personalized Online Course Recommender System with Machine Learning

Su Wu
April 18 2026



Outline

- Introduction and Background
- Exploratory Data Analysis
- Content-based Recommender System using Unsupervised Learning
- Collaborative-filtering based Recommender System using Supervised learning
- Conclusion
- Appendix

Introduction

- **Project background and context**

  Online learning platforms provide thousands of courses, making it difficult for users to find relevant content efficiently. Recommender systems help solve this problem by suggesting personalized courses based on user behavior and course features.

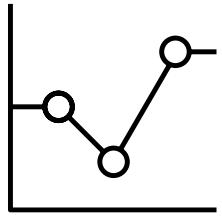
- **Problem Statement**

  The goal of this project is to build a personalized course recommender system using multiple machine learning approaches and compare their performance.

- **Hypotheses**

  1. Content-based methods can recommend relevant courses using course features (genres, titles)
  2. Collaborative filtering methods can capture user preferences more accurately
  3. Neural network embeddings can further improve recommendation quality by learning hidden patterns

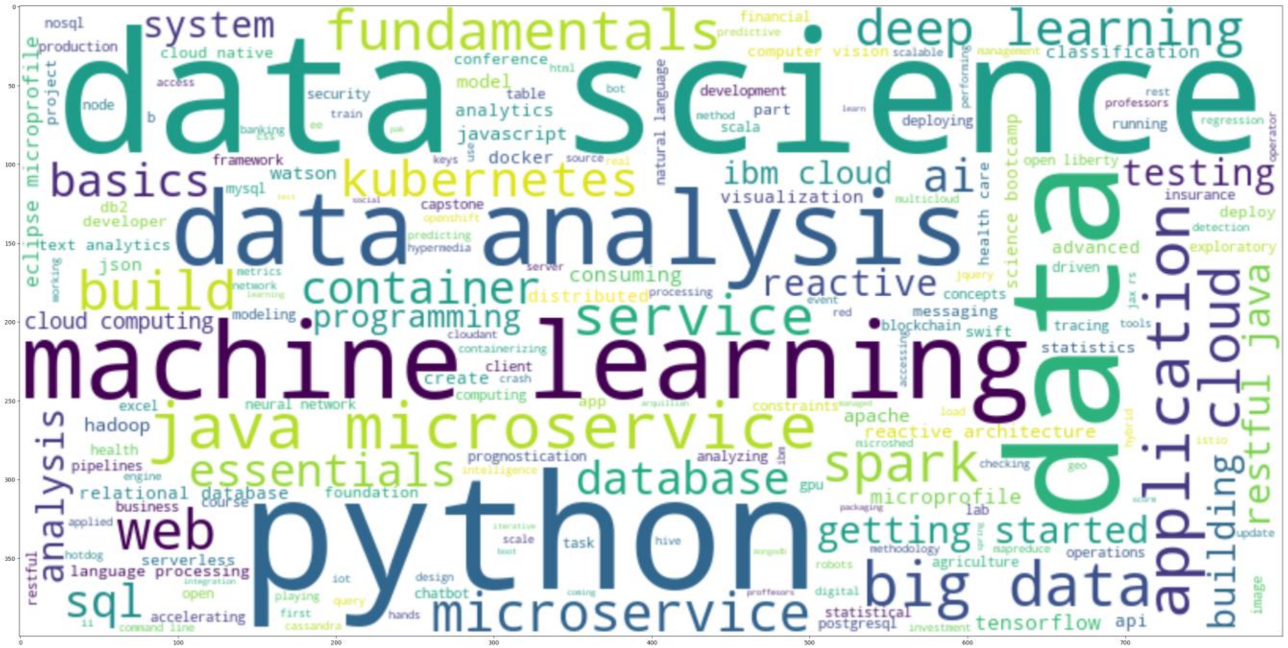
Exploratory Data Analysis



Word cloud of course titles

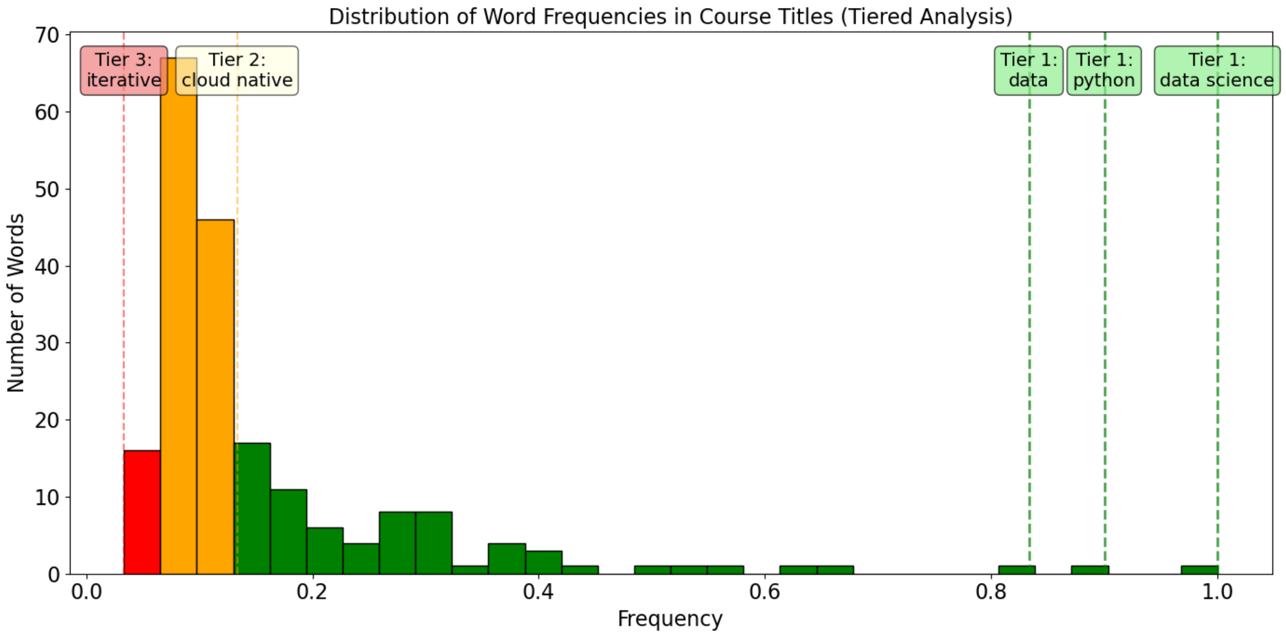
- Frequent keywords in course titles **(A)**: larger = more frequent.
- **Data, Python, and Machine Learning** are the top topics.

A



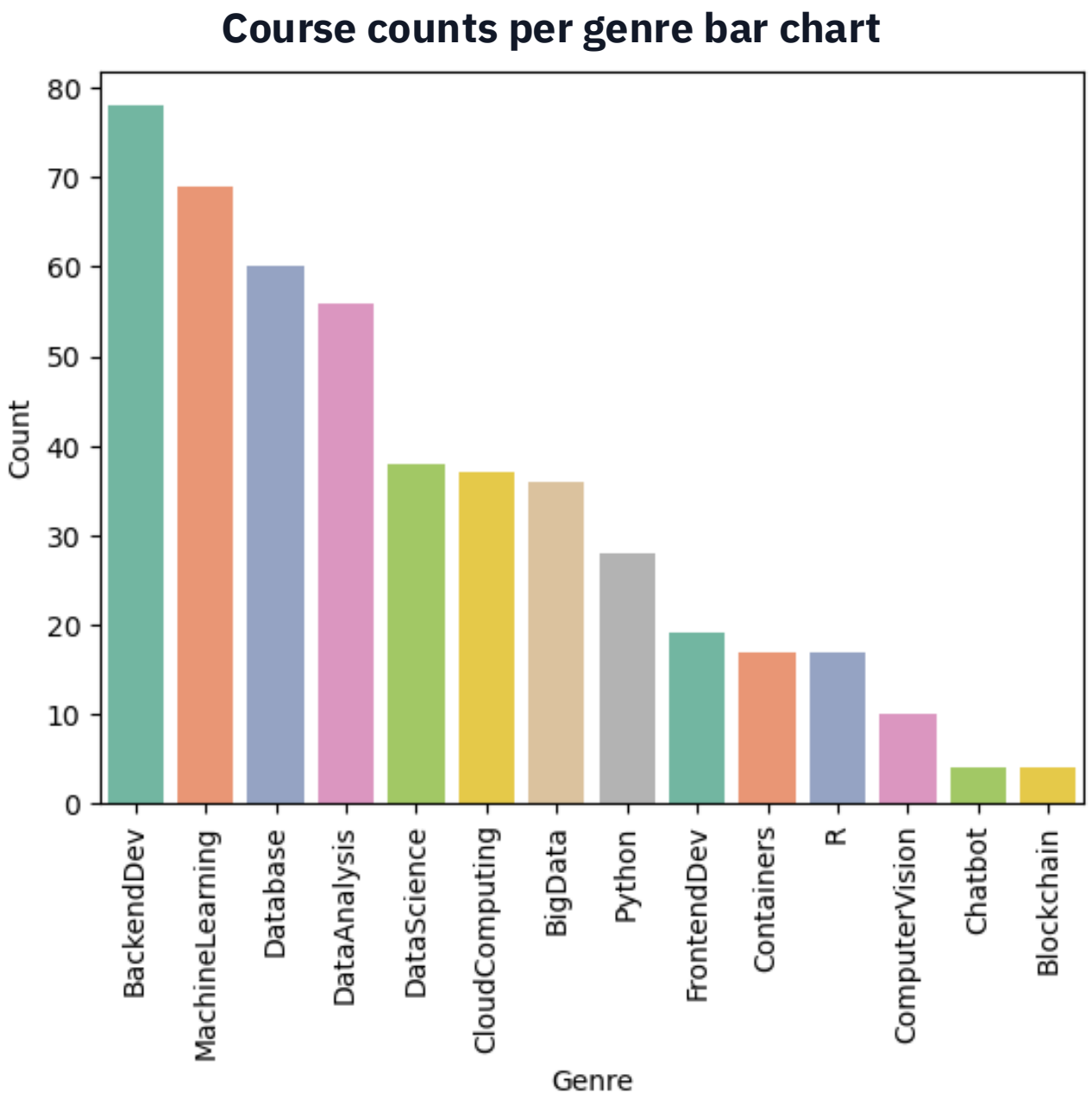
- Word frequency distribution **(B)**:
  - Tier 1: few keywords, very frequent
  - Tier 2: most words, moderate frequency
  - Tier 3: low frequency
- **Few keywords dominate** with strong focus on data science & programming
- Note: X-axis shows relative frequency (1.0 = most frequent word).

B



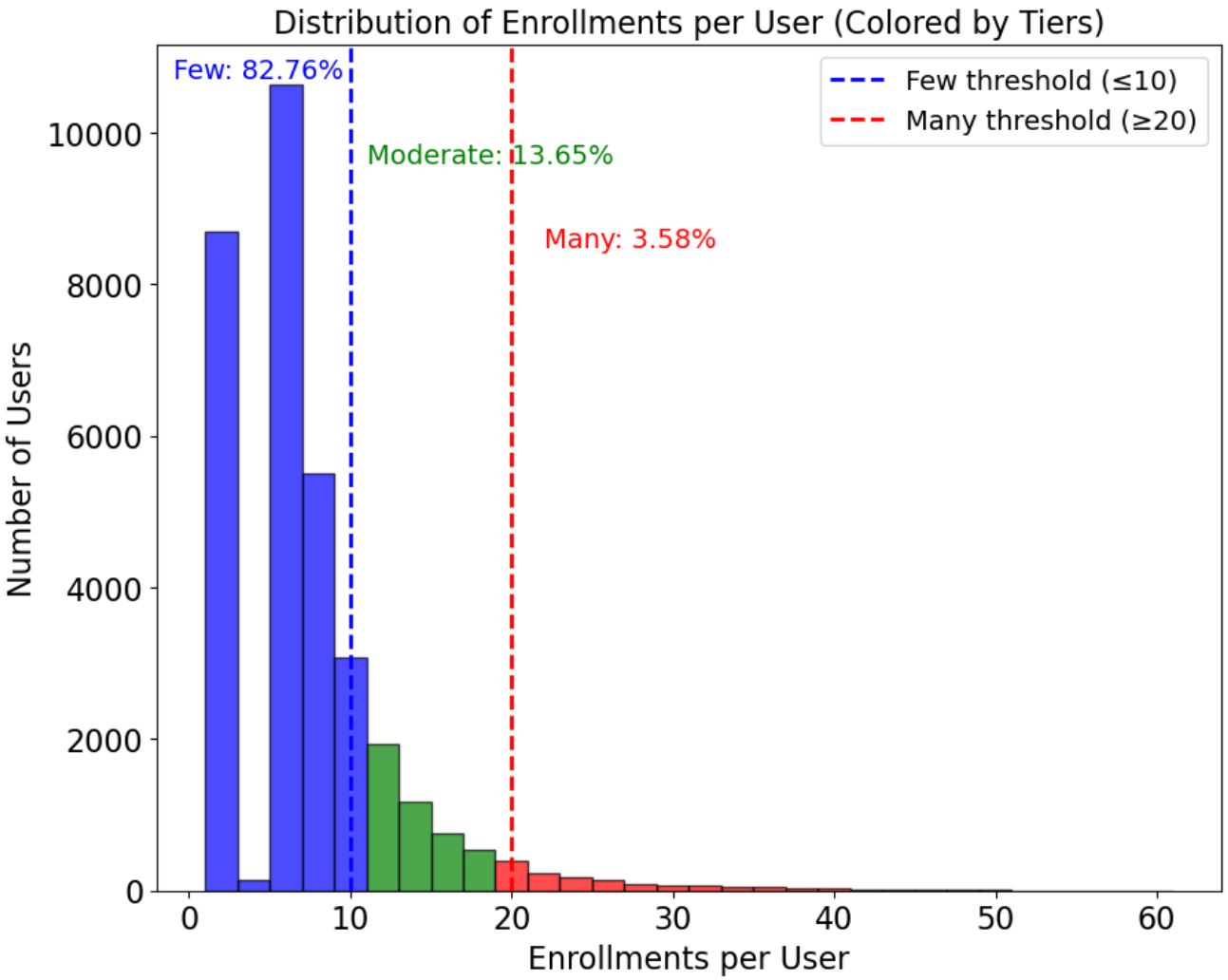
Course counts by genre

- Genres ranking by course count
- Top categories: *Backend Development* and *Machine Learning*.
- Mid-tier: *Database*, *Data Analysis*, and *Data Science*.
- Lower representation: *Blockchain*, *Chatbots*.
- Courses are concentrated in **backend** & **data-related fields**, with limited coverage of emerging topics



User enrollment distribution

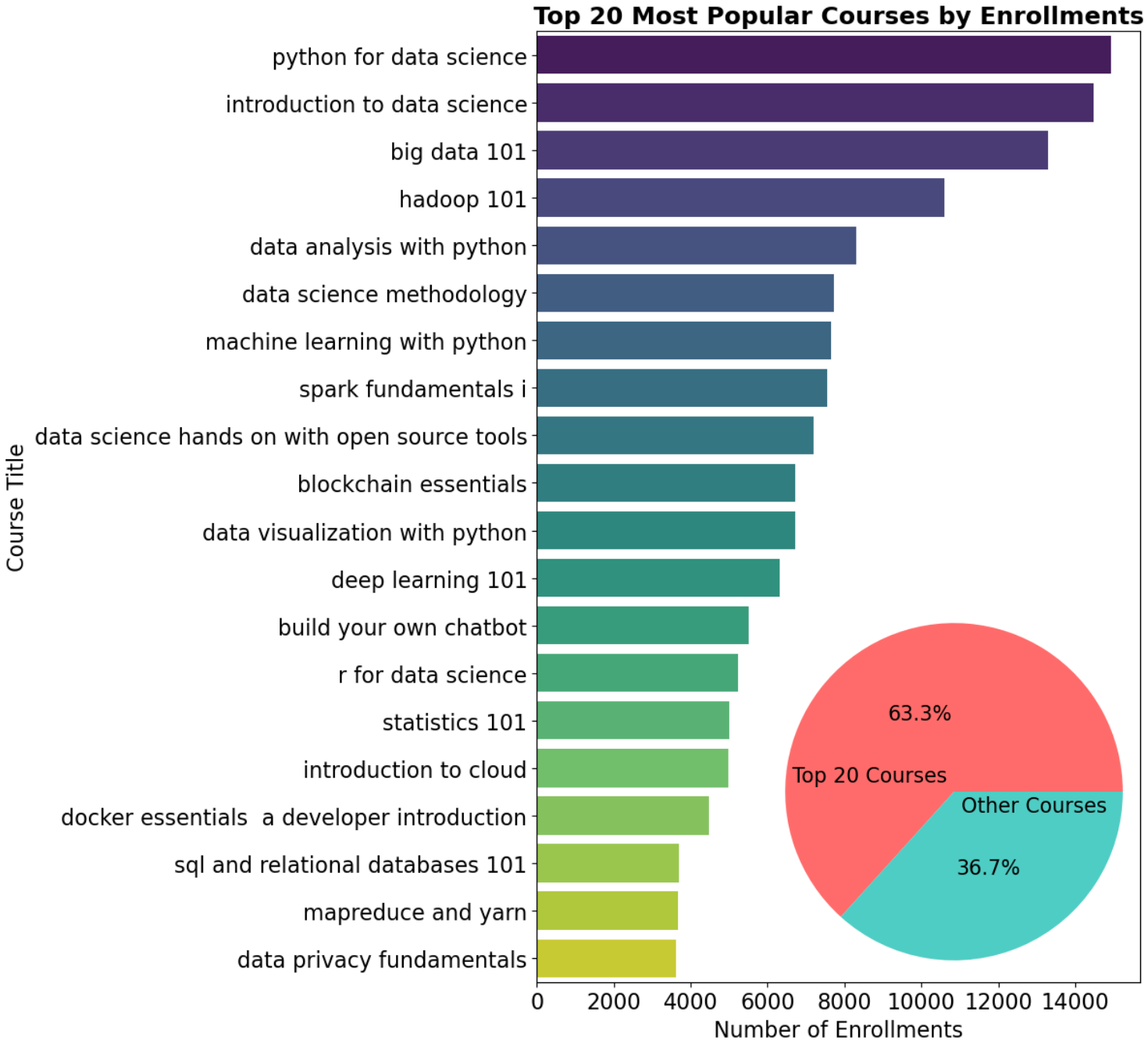
- Most users take few courses: 82.76% enroll in ≤10 courses
- Moderate users: 13.65% enroll in 11–20 courses
- Highly active users are rare: 3.58% enroll in ≥20 courses
- Mean: ~7 courses per user
- Most users show low engagement, while a small group drives high activity



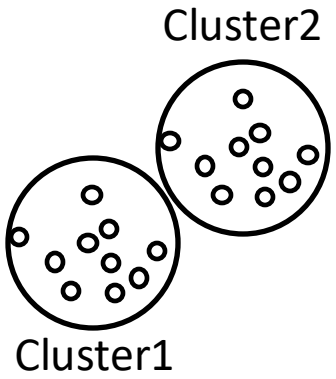
Top 20 most popular courses

- Dominated by **data science and Python-related** topics.
- **Top courses:** Python for Data Science, Introduction to Data Science
- **Common themes**: Data analysis, machine learning, big data (Hadoop, Spark)
- Enrollment is **highly concentrated**: top 20 courses account for 63.30% of total enrollments.

course popularity plot

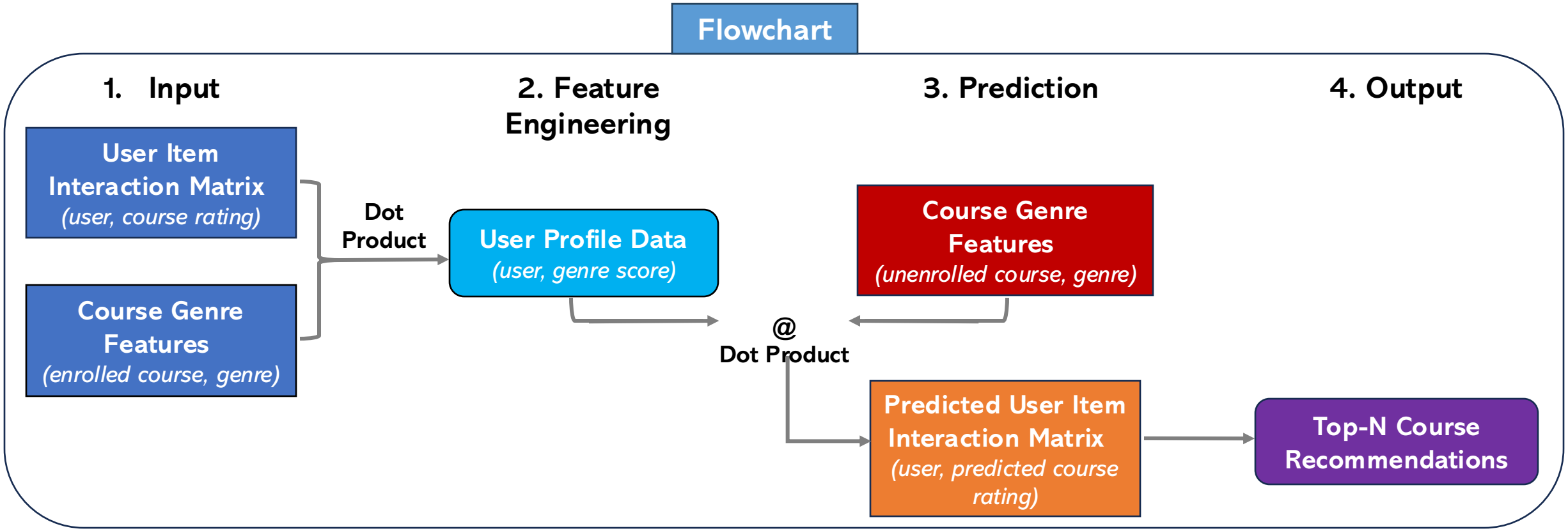


Content-based Recommender System using Unsupervised Learning

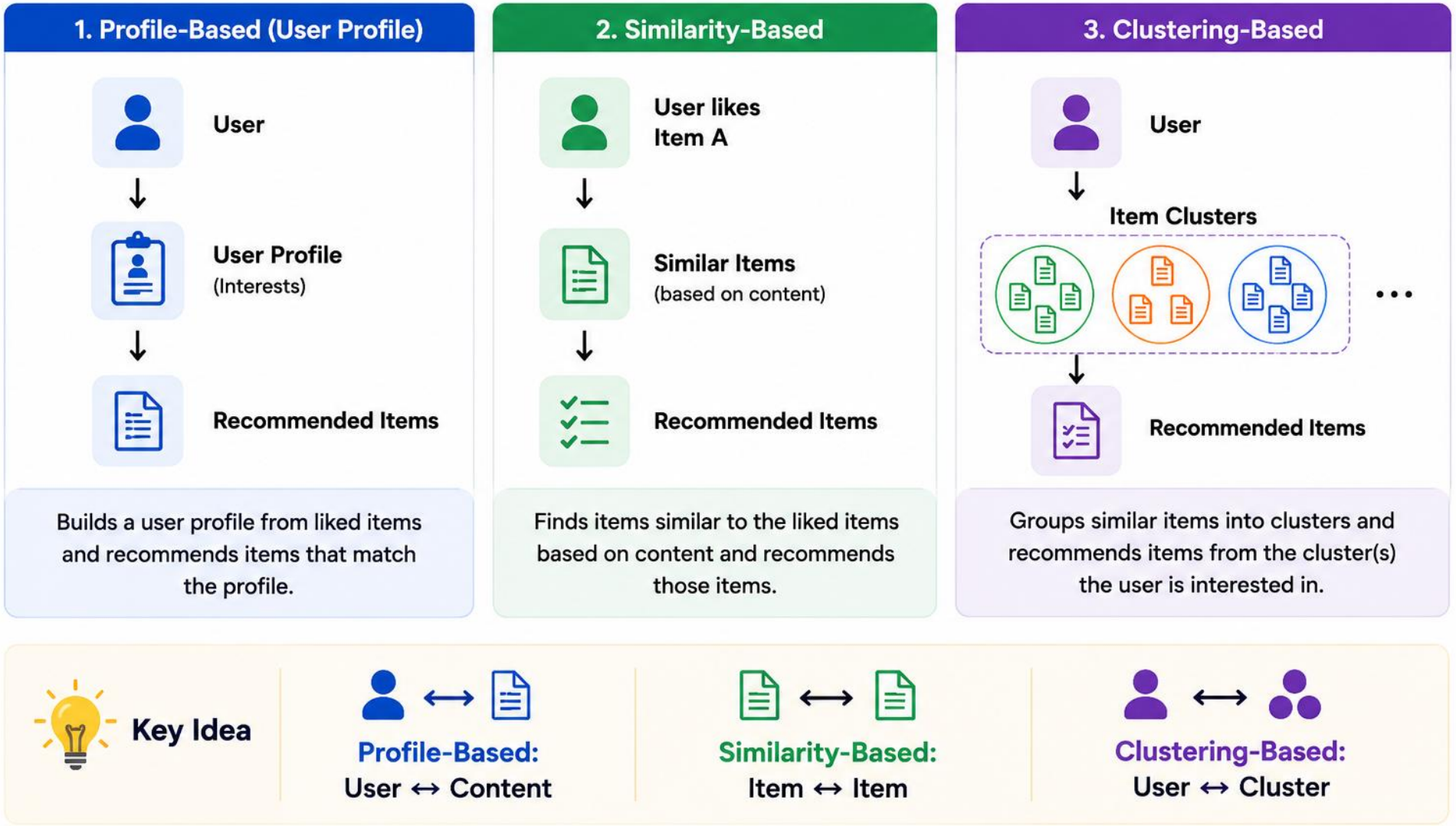


Content-based recommender system using user profile and course genres

- User preferences are derived from past ratings and represented as a genre vector.
- This profile is used to score unseen courses based on genre similarity, and the highest-scoring courses are recommended.



Comparison of content-based recommendation methods

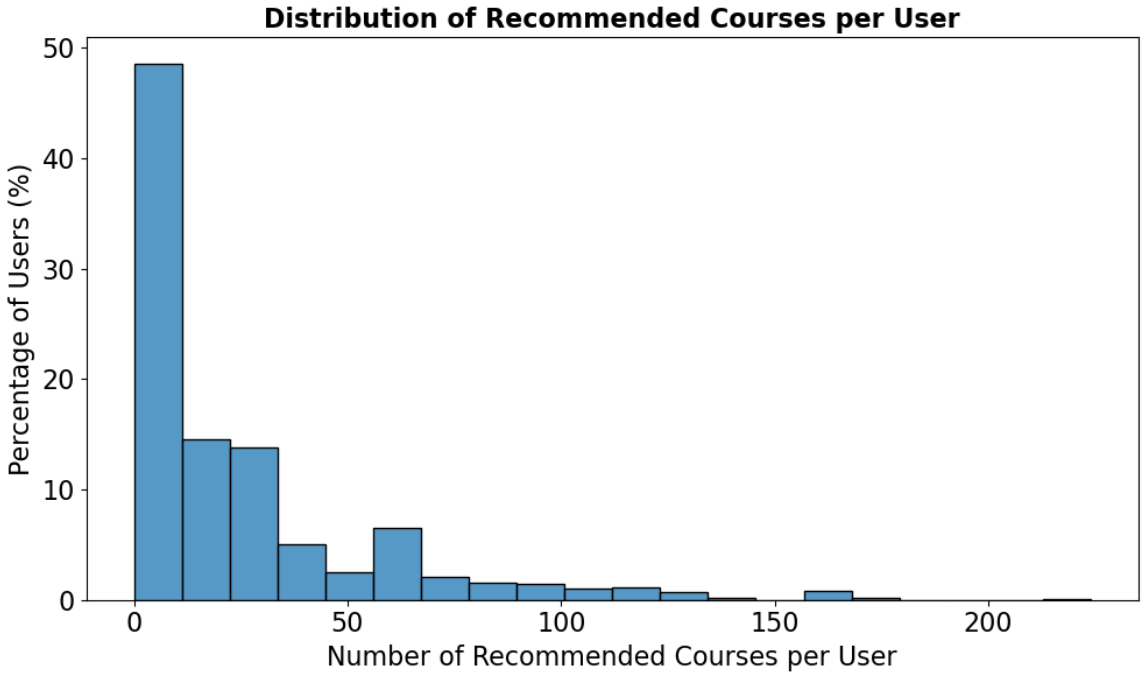


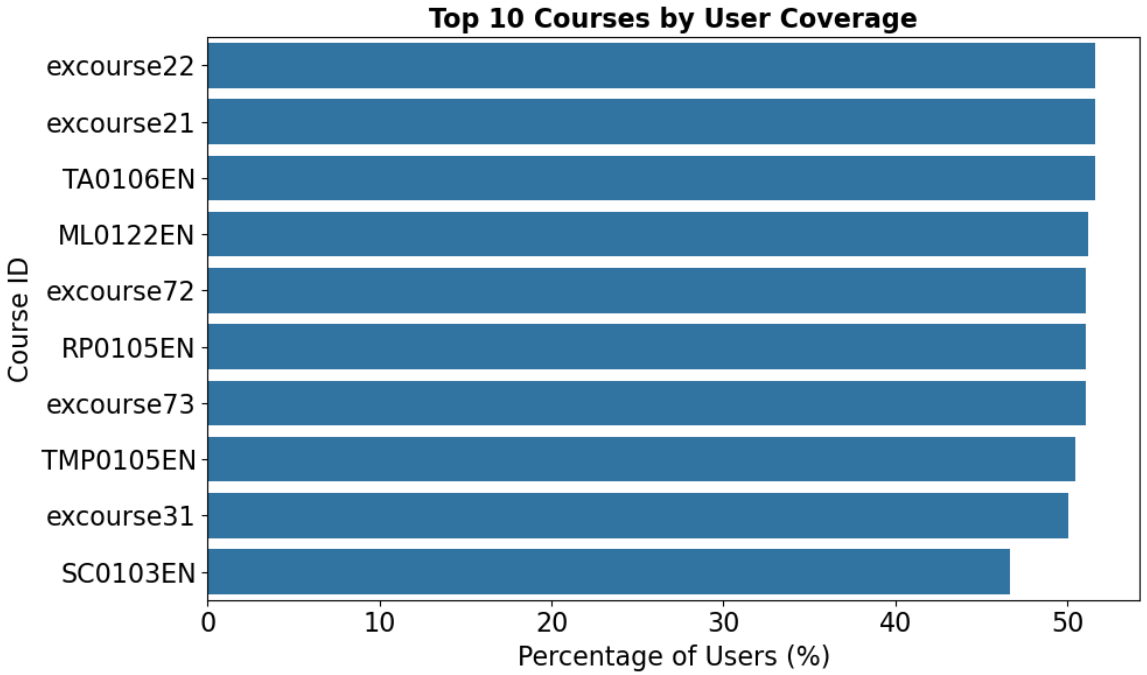
User profile-based recommender system - evaluation

- Score threshold for recommendations: **18**
- Average recommendations: **24.2** courses per user

- Most users receive < **30** recommendations
- Distribution with a long tail (>100 for some users)

- Top 10 courses cover ~**45–55%** of users each
- Recommendations dominated by a small set of popular courses

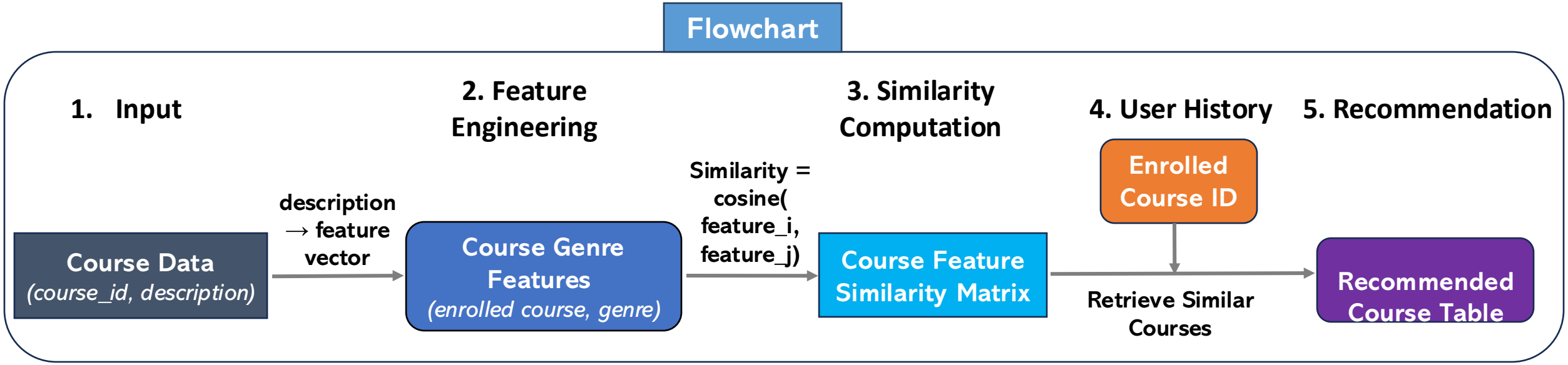




Content-based recommender system via course similarity

- Course descriptions are converted into genre feature vectors using Bag-of-Words (BoW)
- Cosine similaritv identifies similar courses for recommendation.

$$\mathrm{sim}(A, B) = \frac{A \cdot B}{\|A\|\|B\|}$$

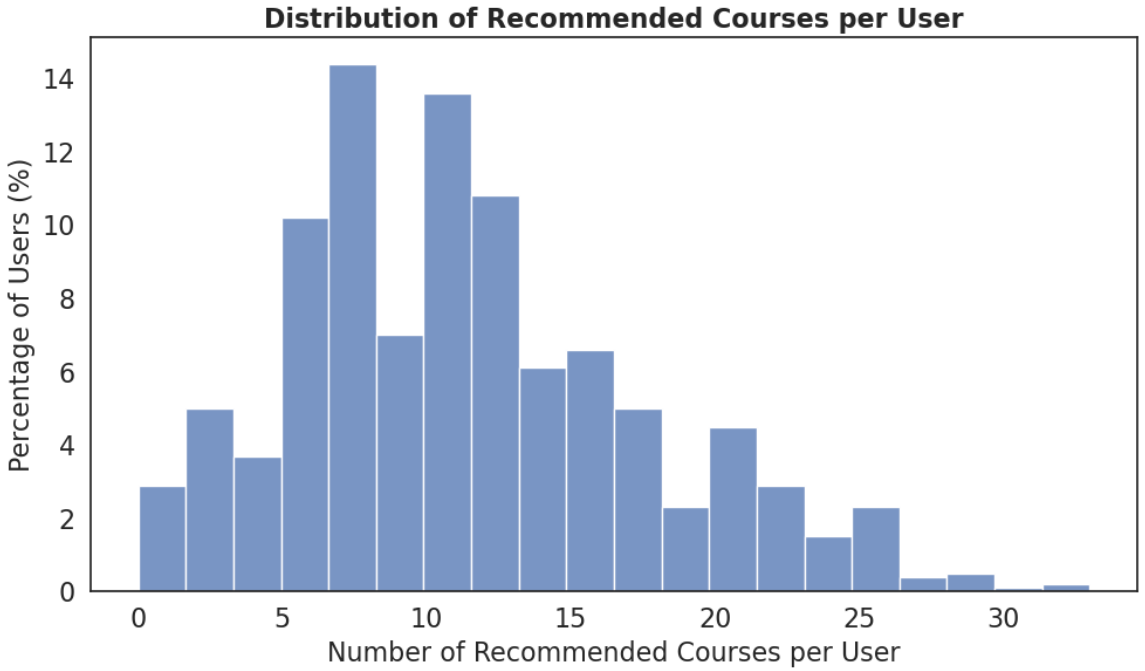


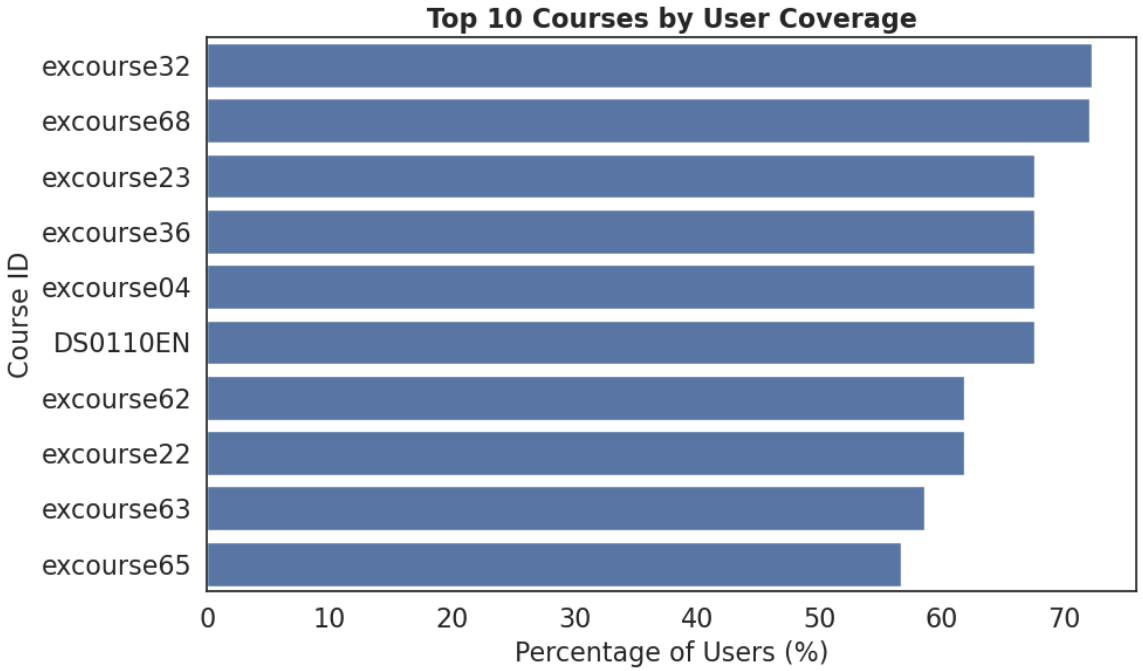
Course similarity-based recommender system - evaluation

- Course similarity threshold: **0.55**
- Average recommendations: **18.6** courses per user

- Most users receive **~5-15** recommendations
- More concentrated distribution → more consistent recommendations

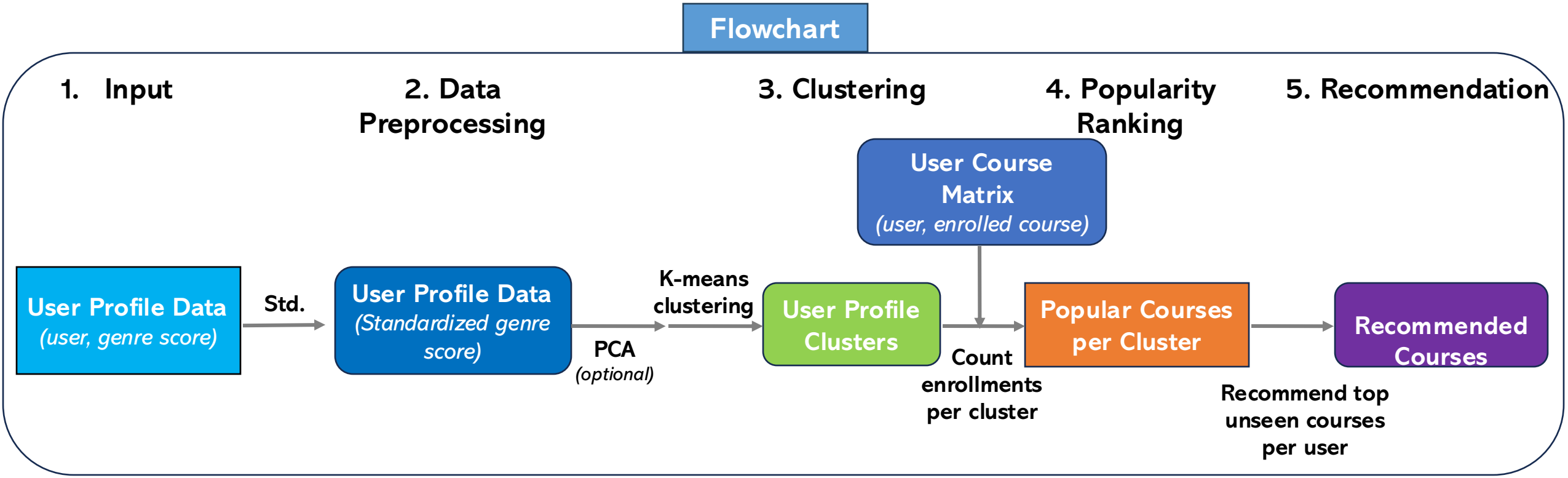
- Top 10 courses reach **~60–70%** of users
- Stronger popularity bias than profile-based method





Clustering-based recommender system

- Cluster users with similar genre interests using **K-means**
- Optionally apply **PCA** to reduce feature number and improve clustering
- Recommend popular **unseen** courses within each cluster

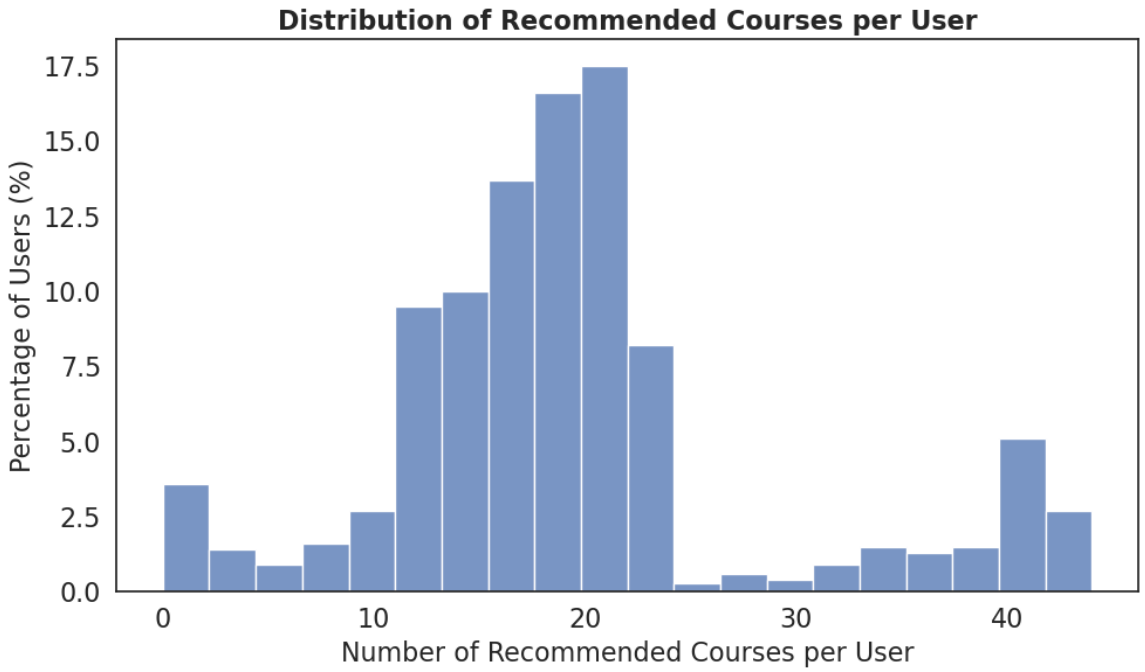


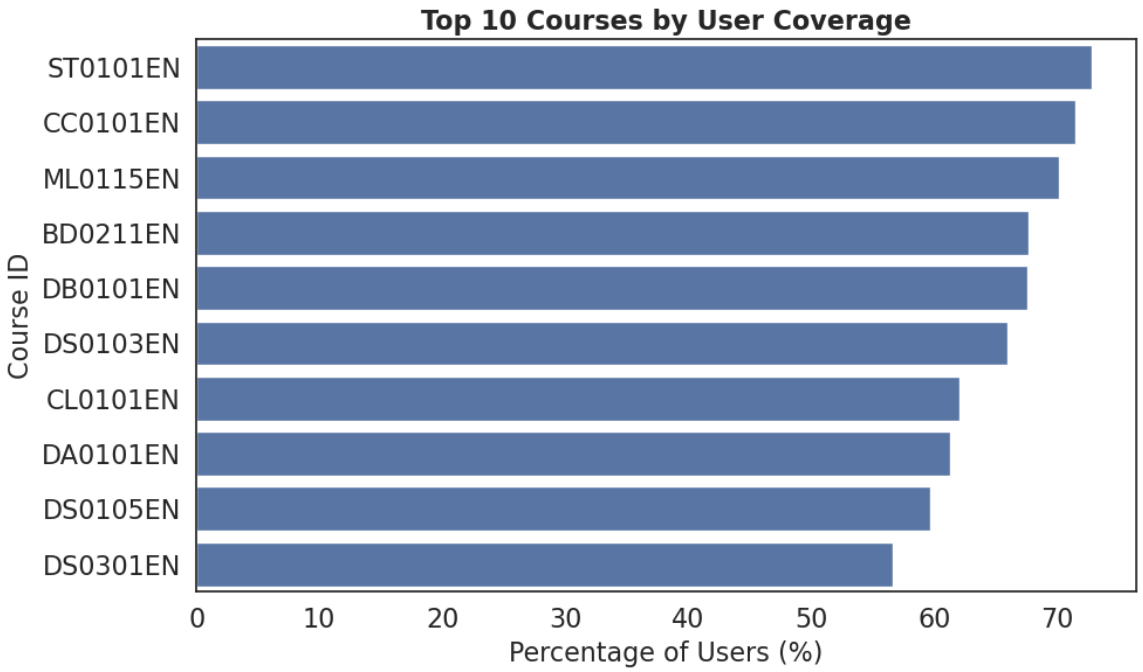
Clustering-based recommender system - evaluation

- Popularity threshold per cluster: **enrollments > 10**
- Average recommendations: **19.3** courses per user

- Most users receive **~10-20** recommendations
- Secondary peak (~35–45) suggests cluster-dependent behavior

- Top 10 courses reach **~60–70%** of users
- High popularity bias and reduced recommendation diversity





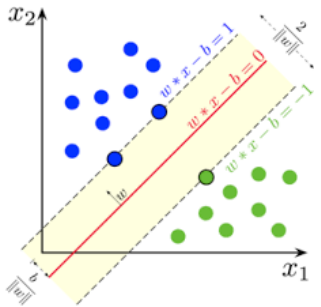
Comparison of content-based recommendation results

Content-based methods recommend items by matching item features with user preferences.

Method	Personalization	Consistency	Diversity	Popularity Bias
Profile-based	⭐⭐⭐⭐ High	⭐⭐ Low	⭐⭐⭐⭐ High	⭐⭐ Low
Similarity-based	⭐⭐⭐ Medium	⭐⭐⭐⭐ High	⭐⭐ Low	⭐⭐⭐⭐ High
Clustering-based	⭐⭐ Low–Medium	⭐⭐⭐⭐ High	⭐⭐ Low	⭐⭐⭐⭐ High

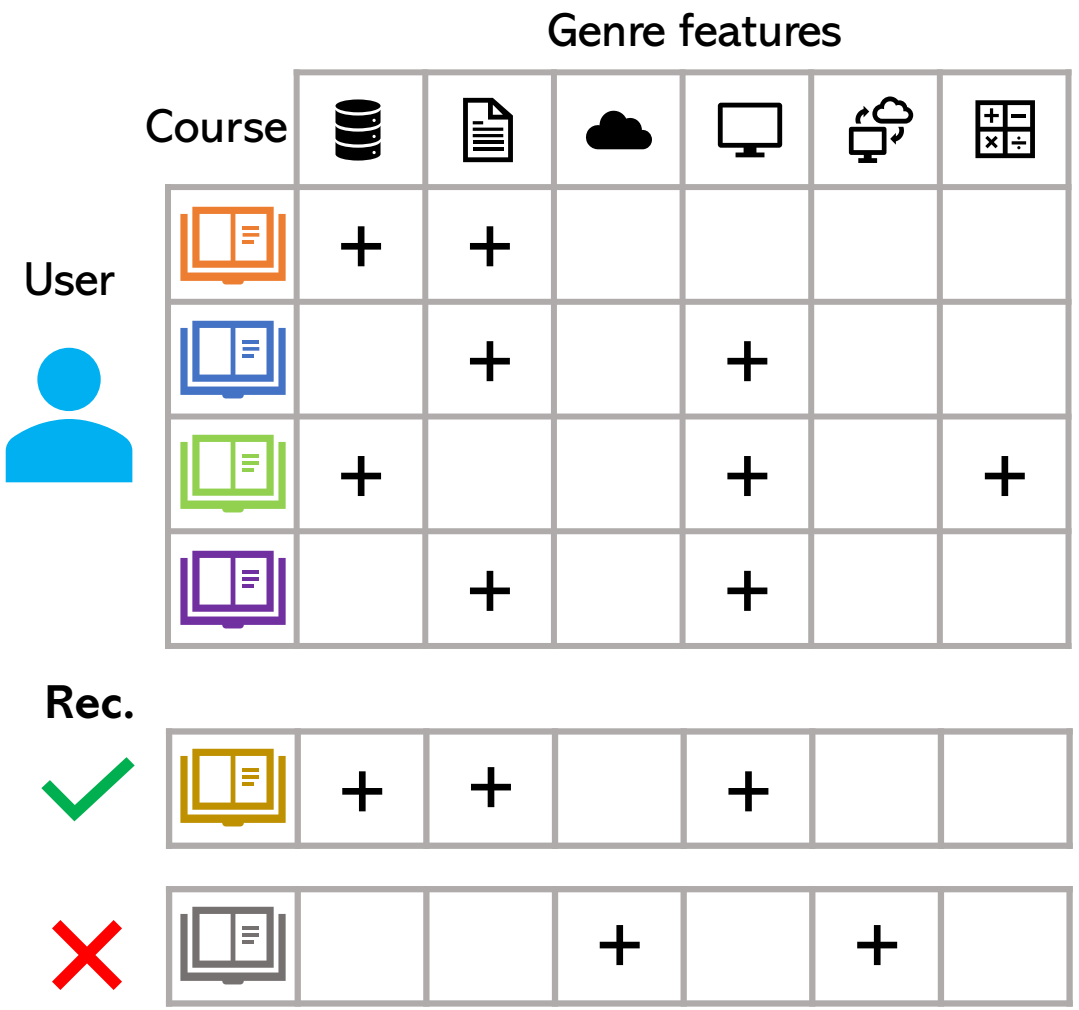
Key Takeaway:
Profile-based maximizes personalization, while similarity- and clustering-based methods increase consistency at the cost of diversity.

Collaborative-filtering Recommender System using Supervised Learning

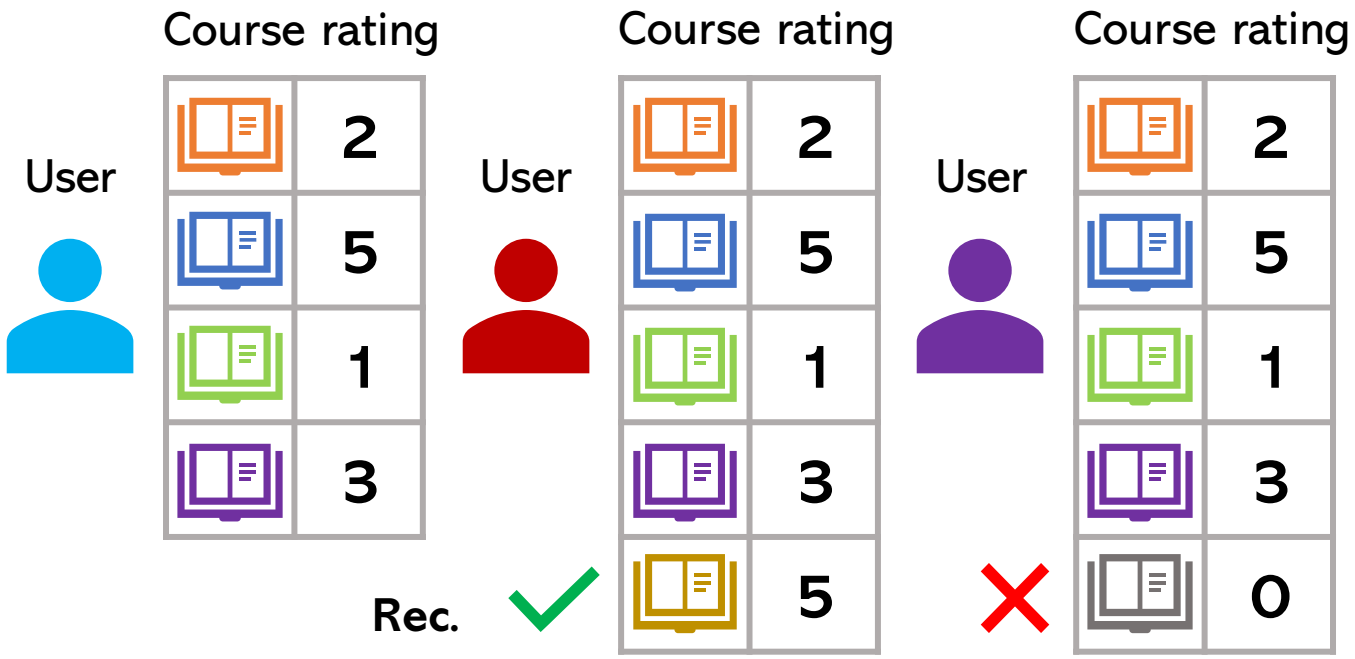


Content-based vs Collaborative filtering

- *Content-based methods recommend items by matching item features with user preferences.*



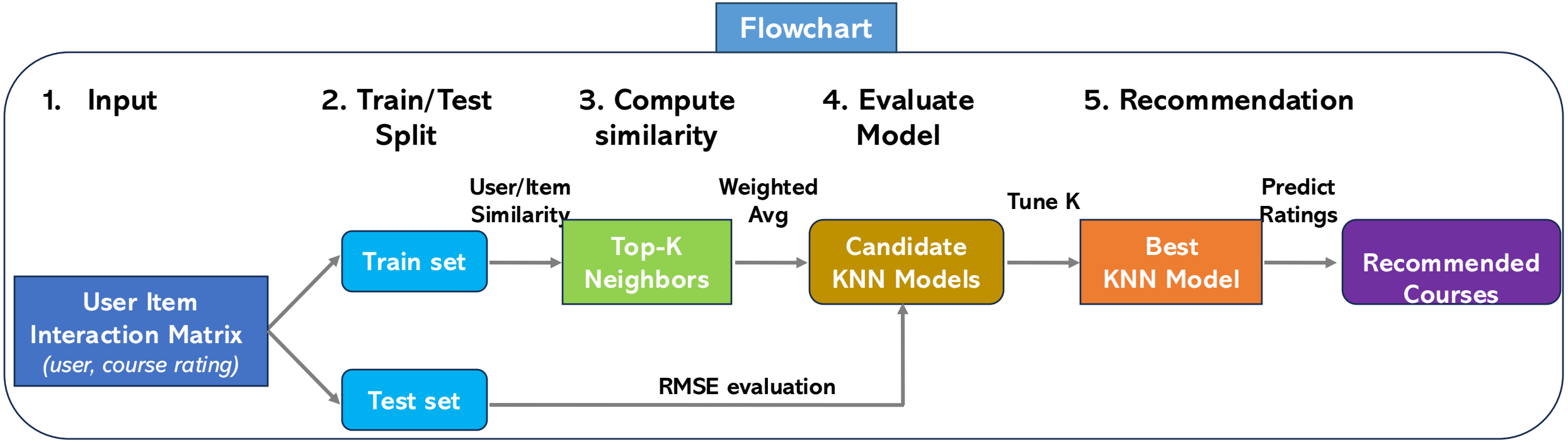
- *Collaborative filtering recommends items based on patterns in user and item interactions.*



Collaborative filtering recommender system using K nearest neighbor (KNN)

- Identify similar users/items using KNN
- Predict missing course ratings via similarity-weighted rating
- Local similarity model; limited by sparsity and scalability

$$\hat{r}_{ui} = \frac{\sum_{v \in N_i^k(u)} \text{similarity}(u, v) \cdot r_{vi}}{\sum_{v \in N_i^k(u)} \text{similarity}(u, v)}$$



KNN Technical Details

Key takeaway:

- Best model: k = 5, MSD, user-based
- RMSE ≈ 0.192

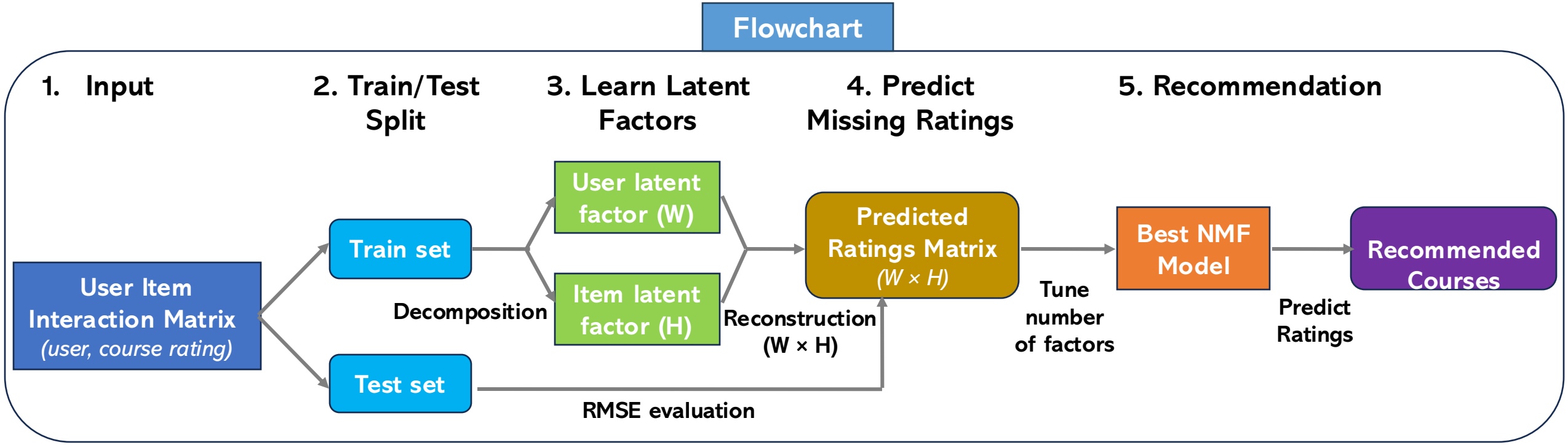
Insights:

- MSD slightly outperforms cosine → MSD uses actual rating differences, Cosine only compares rating patterns
- Small k (5–10) performs best → Large k includes less similar neighbors and adds noise
- User-based works better with MSD, Not consistent with cosine

k	similarity	user_based	val_rmse
5	msd	True	0.1919
10	msd	True	0.1922
20	msd	True	0.1931
5	msd	False	0.1933
10	msd	False	0.1935
20	msd	False	0.1936
20	cosine	False	0.1939
10	cosine	False	0.194
5	cosine	False	0.1941
20	cosine	True	0.2029
10	cosine	True	0.2036
5	cosine	True	0.2044

Collaborative filtering recommender system using Non-negative matrix factorization (NMF)

- NMF decomposes the user–item matrix into latent user and item factors
- Missing ratings are predicted via: $r = W \cdot H$
- Captures global user–item interactions and effectively handles sparse data



NMF Technical Details

Key takeaway:

- Best model: n_factors = 45, epochs = 50
- RMSE ≈ 0.190

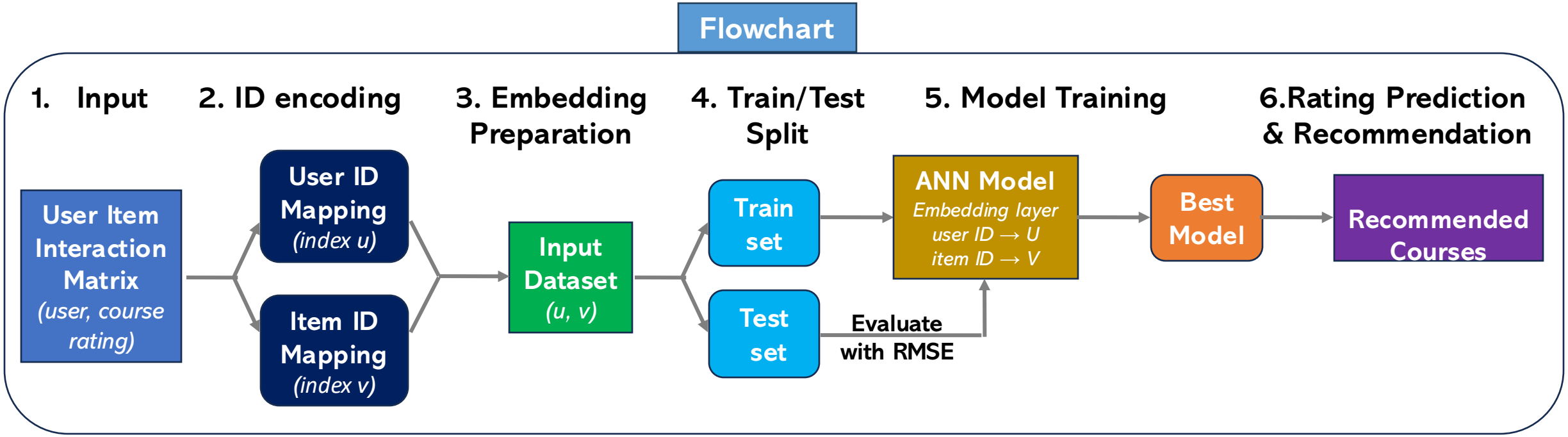
Insights:

- Increasing n_factors improves performance from very low values, then saturates
- More epochs generally help, but depend on model complexity
- Very low n_factors (e.g., 5) → severe underfitting

n_factors	n_epochs	val_rmse
45.0	50.0	0.1902
30.0	50.0	0.1911
15.0	10.0	0.1952
15.0	20.0	0.1962
30.0	20.0	0.198
45.0	20.0	0.2036
15.0	50.0	0.2048
30.0	10.0	0.2075
45.0	10.0	0.2088
5.0	50.0	0.35
5.0	20.0	0.7627
5.0	10.0	0.9665

Collaborative filtering recommender system using artificial neural network (ANN)

- Embeddings encode latent user preferences and item features
- ANN learns user and item embeddings from interaction data to capture latent representations
- Ratings are predicted via a neural network: $r = f(U, V)$
- Captures nonlinear user–item interactions and performs well on sparse data



Collaborative filtering-based recommender system (ANN) - evaluation

Key takeaway:

- Best model: lr = 0.01, emb = 16, act = tanh
- RMSE ≈ 0.0812

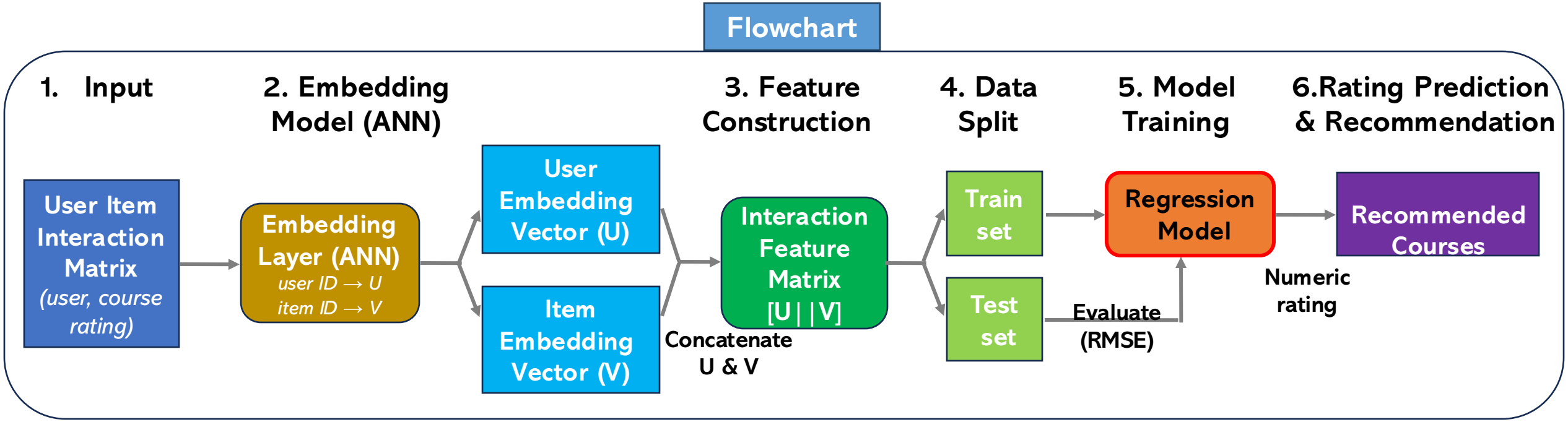
Insights:

- tanh > ReLU → smoother and symmetric outputs, better fit for rating prediction
- Smaller embedding (16) performs best → larger embeddings may overfit on limited data.
- Higher learning rate (0.01) performs better → faster convergence; 0.001 may under-train within limited epochs

lr	emb	act	val_rmse
0.01	16	tanh	0.0812
0.01	32	tanh	0.0825
0.001	32	tanh	0.0882
0.001	16	tanh	0.0922
0.001	32	relu	0.1121
0.001	16	relu	0.1144
0.01	16	relu	0.1151
0.01	32	relu	0.1153

Embedding-based Regression for Rating Prediction

- Learn **user (U) and item (V) embeddings** from ANN
- Construct **interaction features** from embeddings
- Predict ratings using a **linear regression model**
- Combines nonlinear embedding learning with simple regression for efficient prediction on sparse data



Collaborative filtering recommender system using Embedding-based Regression

Key takeaway:

- Best model: Ridge regression
- RMSE ≈ 0.211

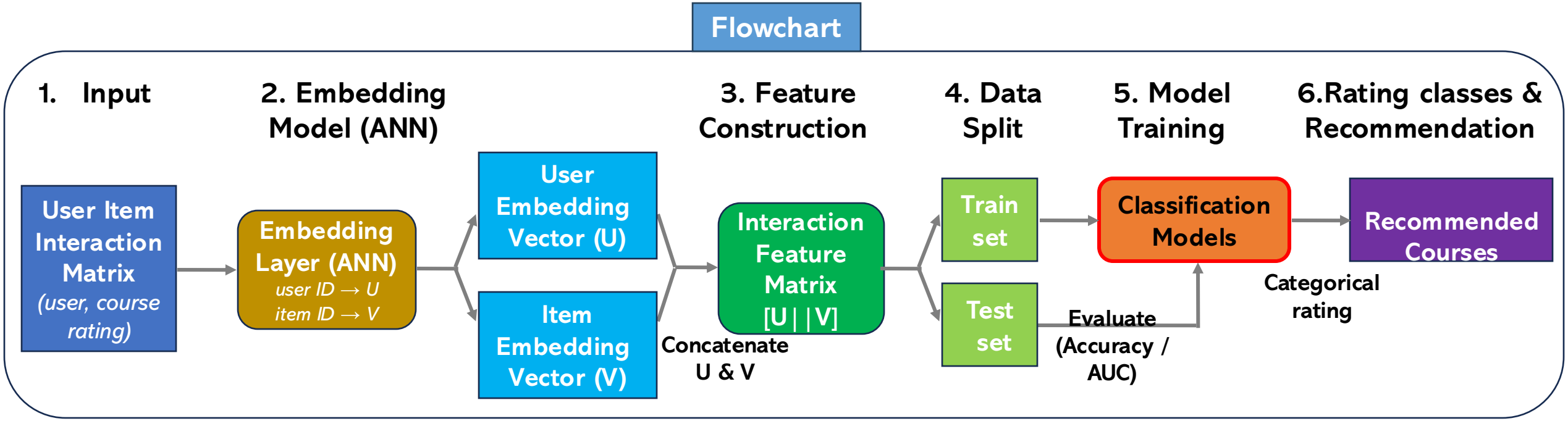
Insights:

- Ridge slightly outperforms Lasso / ElasticNet → embeddings are dense, so shrinking weights works better than removing features
- Changing regularization strength (alpha, l1_ratio) has minimal impact
- All models are linear → cannot capture complex user–item interactions (unlike ANN)

Model	alpha	l1_ratio	val_rmse
Ridge()	0.1	-	0.2107
Ridge()	1.0	-	0.2107
Ridge()	10.0	-	0.2107
Lasso(max_iter=10000)	0.1	-	0.2114
Lasso(max_iter=10000)	1.0	-	0.2114
Lasso(max_iter=10000)	10.0	-	0.2114
ElasticNet(max_iter=10000)	0.1	0.1	0.2114
ElasticNet(max_iter=10000)	0.1	0.9	0.2114
ElasticNet(max_iter=10000)	10.0	0.1	0.2114
ElasticNet(max_iter=10000)	10.0	0.9	0.2114

Embedding-Based Classification for Recommendation

- Learn **user (U) and item (V) embeddings** from ANN
- Construct **interaction features** from embeddings
- Predict ratings classes using classification **models (LR, SVM, RF and XGBoost)**
- Combines nonlinear embedding learning with classification models for efficient prediction on sparse data



Collaborative filtering recommender system using embedding-based classification

Key takeaway:

- Tree-based models outperform linear models:
  - Random Forest → Best overall
  - XGBoost → Strong alternative
- SVM → High accuracy but weak precision
- Logistic Regression → Weak performance

Insights:

- Dataset is highly imbalanced (~95% vs 5%)
- Accuracy is misleading → F1_macro is critical
- Linear models struggle with minority class detection

Model	Accuracy	Precision_weighted	Recall_weighted	F1_weighted	F1_macro	AUC
Random Forest	0.9821	0.9815	0.9821	0.9806	0.8836	0.95
XGBoost	0.9781	0.9783	0.9781	0.9749	0.8433	0.9472
SVM	0.953	0.9081	0.953	0.93	0.488	0.6235
Logistic Regression	0.5784	0.9276	0.5784	0.6942	0.4236	0.6237

Performance comparison of collaborative-filtering models

ANN (0.080) → best performance:

- Captures nonlinear user–item interactions
- Learn complex patterns beyond similarity

KNN, SVD, and NMF perform similarly (RMSE ≈ 0.14–0.21)

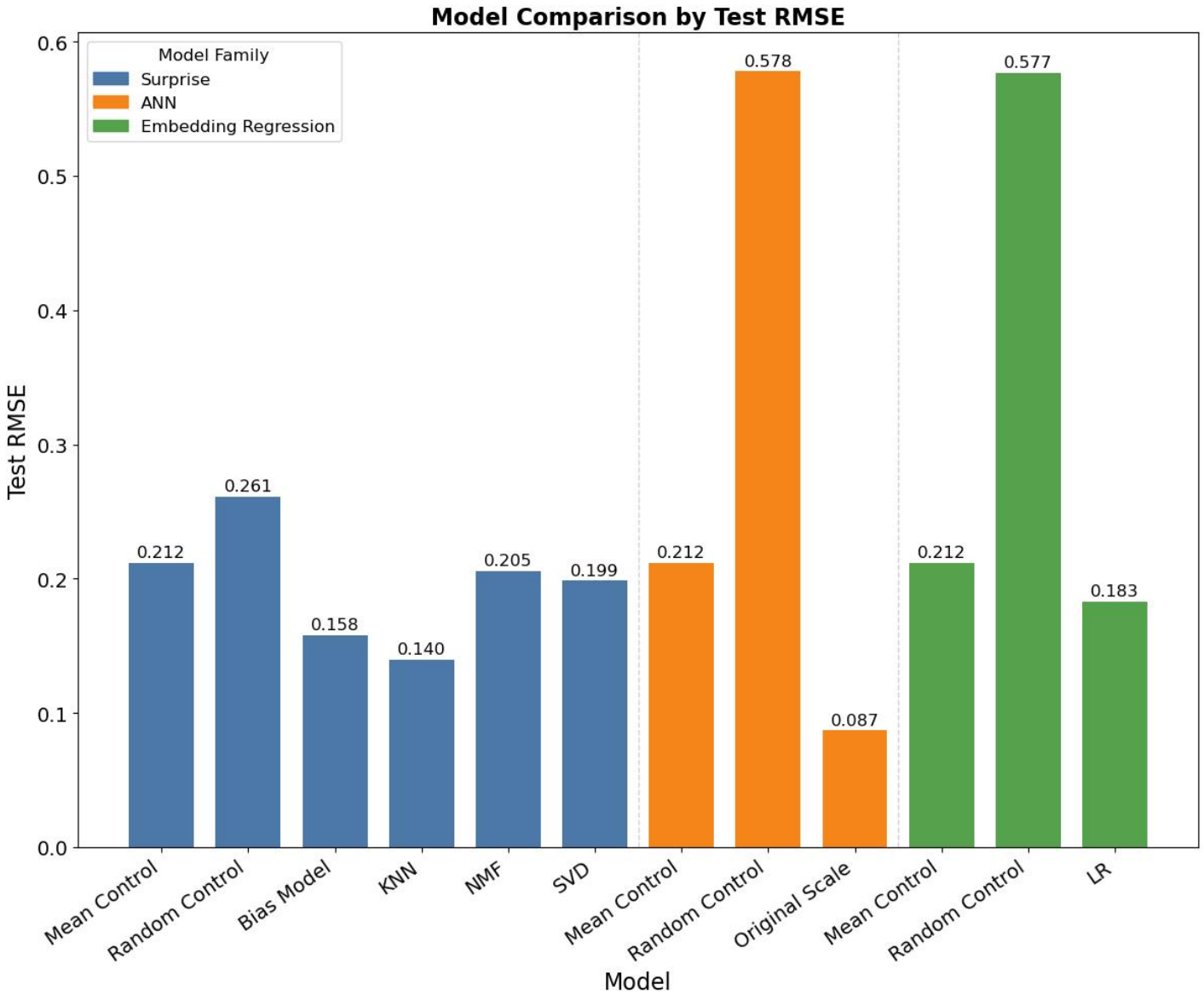
- Based on linear or similarity assumptions
- Limited ability to model higher-order interactions

Mean Control performs well

- Overall rating trends explain much of the signal

Random controls perform poorly

- Models learn meaningful patterns rather than noise



Conclusions

Model Comparison

Content-Based

- Simple, interpretable
- Limited personalization

Collaborative Filtering (KNN, NMF)

- Captures user behavior patterns
- Sensitive to sparse data, higher computation

Neural Network (ANN)

- **Best RMSE** (highest accuracy)
- Learns complex user–item interactions
- Less interpretable

Key Takeaways

- Accuracy vs Interpretability trade-off
- **Popularity bias** observed
  → Many methods favor a few popular courses
- **Data sparsity** limits traditional CF performance
- ANN captures **hidden (latent) structure** better

Recommendation

ANN embedding model is recommended because it captures nonlinear user-item interactions and achieved the best RMSE.